

Escuela de Ingeniería en Computación

Curso: IC-1802 – Introducción a la Programación

Proyecto #2 – Defensa y Asalto de Base

Estudiantes

Génesis Villegas Umaña, 2026018895

Amanda Torres Venegas, 2026018764

Fecha

21 de Junio de 2026

Profesores

Diego Andrés Mora Rojas

Jeff Schmidt Peralta

Descripción General del Juego

Defensa y Asalto de Base es un juego de estrategia para dos jugadores desarrollado en Python con interfaz gráfica Tkinter. Un jugador asume el rol de Defensor y debe proteger una base central construyendo muros y torres defensivas, mientras que el otro jugador es el Atacante y debe destruir dicha base enviando unidades militares a través del mapa.

La partida se desarrolla por rondas. El primer jugador en ganar 3 rondas gana la partida completa. Cada ronda sigue un orden estructurado de fases: construcción del defensor, despliegue del atacante y combate automático.

Flujo General de una Partida

1. Los dos jugadores inician sesión o se registran en el sistema.
2. Cada jugador selecciona una facción diferente (Medieval, Futurista o Naturaleza).
3. Comienza la primera ronda: el Defensor coloca torres y muros en el mapa.
4. El Atacante despliega sus unidades en los bordes del mapa.
5. Se ejecuta el combate automáticamente, turno a turno, hasta que alguien gane la ronda.
6. Se muestra el resultado de la ronda. Si ningún jugador acumula 3 victorias, inicia otra ronda.
7. Al finalizar la partida, se actualiza el registro de victorias del ganador.

Reglas del Juego

Condiciones de Victoria por Ronda

El Defensor gana la ronda si:

- Todas las unidades atacantes son eliminadas.
- El atacante se queda sin dinero y sin unidades activas.
- Se agotan los 100 turnos máximos de combate sin que la base sea destruida.

El Atacante gana la ronda si:

- Logra destruir la base central (reducir su vida a 0).

Victoria de la Partida

El primer jugador que acumule 3 victorias de ronda gana la partida completa. Al terminar, se actualiza el conteo de victorias del jugador ganador según su rol (defensor o atacante) en el archivo JSON de jugadores.

Sistema de Dinero

Ambos jugadores comienzan cada ronda con dinero inicial fijo (\$200). Adicionalmente, al inicio de cada ronda se suma un bono fijo de \$50 a ambos jugadores (a partir de la ronda 2).

El Defensor gana dinero durante el combate:

- \$3 por cada disparo que dañe una unidad enemiga.
- \$10 de bonificación adicional por cada unidad enemiga eliminada.

El Atacante gana dinero durante el combate:

- \$5 cada vez que una unidad llega a atacar la base central.

Restricciones de Posicionamiento

- El Defensor no puede colocar torres ni muros en la casilla (5,5) donde se encuentra la base central.
- El Defensor no puede colocar estructuras encima de otras ya existentes.
- El Atacante solo puede colocar unidades en los bordes del mapa (fila 0, fila 10, columna 0 o columna 10).
- El Atacante no puede colocar unidades encima de otras estructuras.

Clases Implementadas

El proyecto implementa una jerarquía de clases orientada a objetos en el archivo `clases.py`. A continuación se describe cada clase:

1. Clase Jugador

Representa a un usuario registrado en el sistema. Almacena sus credenciales y estadísticas de victorias.

Atributo / Método	Descripción
<code>nombre_usuario</code>	Nombre único de identificación del jugador.
<code>contraseña</code>	Contraseña del jugador.
<code>victorias_defensor</code>	Contador de victorias obtenidas como defensor.
<code>victorias_atacante</code>	Contador de victorias obtenidas como atacante.
<code>sumar_victoria(rol)</code>	Incrementa en 1 la victoria correspondiente al rol indicado (defensor o atacante).
<code>convertir_diccionario()</code>	Convierte los datos del jugador a un diccionario Python para serializar en JSON.
<code>crear_jugador_dicc()</code>	Método estático que construye una instancia Jugador a partir de un diccionario cargado del JSON.

2. Clase Jugadores Json

Gestiona el sistema de registro e inicio de sesión, leyendo y escribiendo en el archivo `data/jugadores.json`.

Método	Descripción
cargar()	Lee el archivo JSON y carga todos los jugadores en memoria como instancias de Jugador.
guardar()	Serializa el diccionario de jugadores en memoria y lo escribe en el archivo JSON.
registrar(usuario, contra)	Verifica que el nombre no exista, crea un nuevo jugador y lo guarda. Retorna (True, mensaje) o (False, error).
iniciar_sesion(usuario, contra)	Valida credenciales. Retorna (True, jugador) si son correctas, o (False, mensaje) si no.
obtener_top_defensores(n)	Retorna los n jugadores con más victorias como defensor, ordenados de mayor a menor.
obtener_top_atacantes(n)	Retorna los n jugadores con más victorias como atacante, ordenados de mayor a menor.

3. Clase Entidad (clase base)

Clase base abstracta de la que heredan Torre, Unidad, Muro y BaseCentral. Centraliza la lógica de puntos de vida y estado de destrucción.

Atributo / Método	Descripción
nombre	Nombre de la entidad.
vida_maxima	Puntos de vida máximos de la entidad.
vida	Puntos de vida actuales.
daño	Daño que inflige la entidad.
recibir_daño(cantidad)	Resta la cantidad indicada a la vida actual, con un mínimo de 0.
esta_destruido()	Retorna True si la vida actual es igual o menor a 0.

4. Clase Torre (hereda de Entidad)

Representa una torre defensiva colocada por el Defensor. Tiene alcance para disparar a unidades y puede activar habilidades especiales según un contador de turnos.

Atributo / Método	Descripción
alcance	Radio en casillas (distancia Manhattan) dentro del cual la torre puede atacar.
habilidad	Nombre de la habilidad especial (string identificador).

turnos_habilidad	Cantidad de turnos necesarios para activar la habilidad.
turnos_restantes	Contador regresivo hasta la próxima activación de la habilidad.
posicion	Tupla (fila, columna) indicando la ubicación de la torre en el mapa.
en_rango(fila, col)	Verifica si una posición objetivo está dentro del alcance de la torre usando distancia Manhattan.
activar_habilidad(juego, obj)	Ejecuta la habilidad especial correspondiente al tipo de torre. Reinicia el contador de turnos.

5. Clase Unidad (hereda de Entidad)

Representa una unidad atacante desplegada por el Atacante. Se mueve turno a turno hacia la base central y puede atacar torres o muros que bloqueen su camino.

Atributo / Método	Descripción
velocidad	Número de casillas que se mueve por turno.
congelada / turnos_congelada	Estado de congelamiento: si está congelada, no puede moverse durante los turnos indicados.
escudo	Puntos de escudo que absorben daño antes de afectar la vida.
mover(destino)	Mueve la unidad a la casilla destino. Si está congelada, reduce el contador y no se mueve.
recibir_daño(cantidad)	Primero absorbe daño con el escudo; el daño restante se aplica a la vida mediante la clase padre.
activar_habilidad(juego, obj)	Ejecuta la habilidad especial según el tipo de unidad. Reinicia el contador de turnos.

6. Clases Muro y BaseCentral (heredan de Entidad)

Clase	Descripción
Muro	Entidad defensiva pasiva con 50 puntos de vida y costo de \$20. Bloquea el movimiento de las unidades atacantes.
BaseCentral	Entidad fija ubicada en la posición (5,5) del mapa con 200 puntos de vida. El atacante debe reducirla a 0 para ganar la ronda.

7. Clase EstadoJuego (en combate.py)

Agrupar todo el estado de una partida en curso: listas de entidades, dinero de cada jugador, marcador de rondas, y la matriz 11x11 del mapa.

Atributo	Descripción
defensor / atacante	Referencias a las instancias Jugador de cada participante.
dinero_defensor / dinero_atacante	Saldo actual de cada jugador.
rondas_defensor / rondas_atacante	Contador de rondas ganadas por cada jugador.
torres / muros / unidades	Listas con las instancias activas de cada tipo de entidad.
base	Instancia de BaseCentral ubicada en (5,5).
mapa	Matriz 11x11 donde cada celda contiene None o una instancia de entidad.

Torres Defensivas

El Defensor puede construir tres tipos de torres. Todas heredan de la clase Torre y comparten la mecánica de habilidades por contador de turnos.

Torre	Costo	Vida	Daño	Alcance	Habilidad
Torre Básica	\$50	80	15	2	Disparo Doble
Torre Pesada	\$120	180	25	2	Daño en Área
Torre Mágica	\$90	60	10	3	Congelar Unidad

Habilidades de Torres

Disparo Doble (Torre Básica):

Cuando el contador llega a 0, la torre inflige una segunda vez su daño base al mismo objetivo que acaba de atacar en ese turno.

Daño en Área (Torre Pesada):

Afecta a todas las unidades dentro del rango de la torre, infligiendo la mitad del daño base a cada una.

Congelar (Torre Mágica):

Congela a la unidad objetivo durante 2 turnos. Mientras está congelada, la unidad no puede moverse.

Unidades Atacantes

El Atacante puede desplegar tres tipos de unidades. Cada una hereda de la clase Unidad y tiene una habilidad especial propia.

Unidad	Costo	Vida	Daño	Velocidad	Habilidad
Soldado	\$40	60	18	1	Ataque Doble
Tanque	\$100	200	28	1	Escudo Temporal
Unidad Rápida	\$60	50	12	2	Aumento Velocidad

Habilidades de Unidades

Ataque Doble (Soldado):

Al activarse, el soldado inflige su daño base una segunda vez al objetivo en ese turno.

Escudo Temporal (Tanque):

El tanque gana 30 puntos de escudo que absorben daño antes de que afecte su vida. Si el escudo es insuficiente, el daño restante se aplica directamente a la vida.

Aumento de Velocidad (Unidad Rápida):

Al activarse, la velocidad de la unidad se incrementa en 1 (de 2 a 3 casillas por turno) de forma permanente para el resto de la ronda.

Facciones

Antes de iniciar la partida, cada jugador elige una facción. Las facciones determinan el aspecto visual de las torres, muros, unidades y la base central. El defensor y el atacante no pueden usar la misma facción.

Facción	Color representativo	Descripción
Medieval	#986846 (marrón cálido)	Estética de castillos y caballería. Torres de piedra, muros amurallados.
Futurista	#4F8D94 (cian metálico)	Estética tecnológica. Torres láser, muros de metal, unidades robóticas.
Naturaleza	#7EB758 (verde)	Estética orgánica. Torres de madera y enredaderas, unidades con camuflaje.

Cada facción tiene un set de imágenes PNG correspondiente para torres (N1, N2, N3), muros, unidades (soldado, tanque, rápida) y la base, cargadas al inicio del programa desde la

carpeta imagenes/. Estas imágenes fueron generadas con ayuda de la inteligencia artificial Gemini.

Mapa del Juego

El mapa es una cuadrícula de 11x11 casillas representada internamente como una matriz bidimensional de Python (lista de listas). Cada celda puede contener None o una instancia de alguna entidad.

- Dimensiones: 11 filas x 11 columnas (121 casillas en total).
- La base central siempre ocupa la posición (fila=5, columna=5), es decir, el centro exacto.
- Cada casilla mide 64x64 píxeles en la interfaz gráfica, dando un canvas de 704x704 píxeles.
- El mapa se dibuja usando un Canvas de Tkinter, redibujado completo en cada turno de combate.

Inicialización del mapa:

Al crear una instancia de EstadoJuego, la matriz se llena con None en todas las posiciones y se coloca el objeto BaseCentral en (5,5). Cuando se coloca una entidad, su posición en la matriz se actualiza; cuando es eliminada, esa posición vuelve a None.

Motor de Combate

La lógica de combate está íntegramente en combate.py. El combate es automático y se ejecuta turno a turno hasta un máximo de 100 turnos.

Función ejecutar_turno(estado)

Se ejecuta una vez por turno y realiza las siguientes acciones en orden:

1. Movimiento de unidades: cada unidad se mueve tantas casillas como indica su velocidad.
2. Ataque a obstáculos: si una unidad encuentra una torre o muro bloqueando su camino, lo ataca con su daño. Si es destruido, se elimina del mapa.
3. Ataque a la base: si la unidad llega a la casilla (5,5), ataca la base central.
4. Disparo de torres: cada torre busca la primera unidad dentro de su alcance y le dispara. Acumula dinero para el defensor por cada disparo y por cada eliminación.
5. Habilidades: el contador de turnos de cada torre disminuye en 1 por turno. Al llegar a 0, se activa la habilidad y el contador se reinicia.
6. Limpieza: las unidades destruidas se eliminan de la lista y del mapa.

Función calcular_movimiento(unidad, mapa)

Determina la siguiente casilla a la que debe moverse una unidad. La lógica prioriza alinearse primero en columna con la base (columna 5) y luego avanzar en fila. Si la casilla frontal está ocupada por una torre o muro, retorna None para que la unidad ataque en vez de moverse.

Función verificar_fin_ronda(estado)

Se llama después de cada turno. Evalúa tres condiciones: si la base fue destruida (victoria atacante), si no quedan unidades vivas (victoria defensor) y si el atacante no tiene dinero ni unidades (victoria defensor). Retorna el rol del ganador o None si la ronda continúa.

Función preparar_nueva_ronda(estado)

Limpia el estado del tablero para la siguiente ronda: elimina todas las unidades, torres y muros del mapa, restaura la vida de la base central a su máximo, y suma el dinero de ronda (\$50) a ambos jugadores.

Sistema de Archivos

El proyecto utiliza un único archivo de datos persistente:

Archivo	Ruta	Descripción
jugadores.json	data/jugadores.json	Diccionario JSON con todos los jugadores registrados y sus estadísticas de victorias.

Estructura del archivo jugadores.json

El archivo contiene un objeto JSON donde cada clave es el nombre de usuario, y su valor es un objeto con los campos: nombre_usuario, contraseña, victorias_defensor y victorias_atacante. Por ejemplo:

```
{ "Genesis": { "nombre_usuario": "Genesis", "contraseña": "123123",  
"victorias_defensor": 4, "victorias_atacante": 3 } }
```

La clase Jugadores_Json se encarga de leer este archivo al iniciar el programa (método cargar) y de escribirlo cada vez que se registra un nuevo jugador o finaliza una partida (método guardar). Si el archivo no existe, se crea automáticamente al primer registro.

Estructuras de Datos Utilizadas

Estructura	Uso en el proyecto	Detalles
Lista (list)	estado.torres, estado.muros, estado.unidades	Almacenamiento dinámico de entidades activas. Permite agregar y eliminar elementos en tiempo de ejecución.
Matriz 11x11 (list of lists)	estado.mapa	Representación espacial del tablero. Permite acceso O(1) a cualquier posición por coordenadas (fila, col).

Diccionario (dict)	TORRES, UNIDADES, FACCIONES, jugadores{}	Acceso rápido por clave. Usado para constantes del juego y para el registro de jugadores en memoria.
Tupla (tuple)	entidad.posicion	Almacenamiento inmutable de coordenadas (fila, columna). Garantiza que la posición no sea modificada accidentalmente.
JSON (archivo)	data/jugadores.json	Persistencia de datos de usuarios entre sesiones del programa.

Arquitectura del Proyecto

Archivo	Responsabilidad
principal.py	Punto de entrada del programa. Inicializa la ventana Tkinter, carga todas las imágenes y llama a mostrar_login().
clases.py	Define todas las clases del dominio: Jugador, Jugadores_Json, Entidad, Torre, Unidad, Muro y BaseCentral. También contiene las constantes del juego (TORRES, UNIDADES, FACCIONES, dineros iniciales).
combate.py	Contiene EstadoJuego y todas las funciones del motor de combate: calcular_movimiento, ejecutar_turno, verificar_fin_ronda, preparar_nueva_ronda y ejecutar_combate.
ventanas.py	Gestiona toda la interfaz gráfica Tkinter: login, selección de facciones, mapa del defensor, mapa del atacante, combate animado, resultado de ronda, ganador de partida y top de jugadores.
data/jugadores.json	Almacenamiento persistente de cuentas de usuario y estadísticas.
imagenes/	Carpeta con todos los sprites PNG del juego organizados por tipo y facción.

A lo largo de la elaboración de este proyecto, se utilizó el apoyo de Claude, para aconsejar con la corrección de bugs y la mejora de la estructura de los programas.

Conclusiones

El proyecto Defensa y Asalto de Base logra implementar un juego de estrategia por turnos funcional cumpliendo con los requisitos del enunciado. La arquitectura modular en cuatro archivos (principal, clases, combate, ventanas) facilita el mantenimiento y la comprensión del código.

La herencia de la clase Entidad evita duplicación de lógica de vida y daño entre torres, unidades, muros y la base. El motor de combate en combate.py está completamente desacoplado de la interfaz gráfica, lo que permite probarlo y modificarlo de forma

independiente. La comunicación entre ambas capas se realiza mediante la función callback actualizar_ui pasada como parámetro.

El sistema de archivos JSON garantiza persistencia de usuarios y estadísticas entre sesiones, y el diseño de las clases Jugador y Jugadores_Json encapsula correctamente la lógica de autenticación y ranking.

Video Evidencia Evolución Proyecto

<https://youtu.be/K8muO7beusQ>

Repositorio Github

<https://github.com/gevium/proyecto-III>

Referencias

Anthropic. (2026). Claude (Sonnet 4.6) [Modelo de lenguaje extenso]. <https://claude.ai>

Free Music. (2019, November 19). *Joshua McLean - Mountain Trials* 🎵 NO COPYRIGHT 8-bit
Music [Video]. YouTube. https://www.youtube.com/watch?v=L_OYo2RS8iU

Google. (2026). Gemini [Modelo de lenguaje de gran tamaño]. <https://gemini.google.com/>